

Sign Language Recognition Using CNN

Michael Tesis
AI 700-001
Long Island University
Brooklyn, New York, USA
Michael.Tesis@my.liu.edu

Robert Mullis
AI 700-001
Long Island University
Brooklyn, New York, USA
Robert.Mullis@my.liu.edu

Abstract — From an image containing a letter in American Sign Language, we use a CNN model to recognize the letter in the image and output it as text. Our CNN model preprocesses the image in the convolutional layers with self-learning filters, padding, and pooling. Then the training is completed in the deep neural network. The model performs multi-class classification to predict the best letter candidate for a given image. We demonstrate the capabilities of the chosen Deep Learning algorithm to perform SLR with a high accuracy.

Keywords — SLR, CNN, Deep Learning, filters, pooling

I. INTRODUCTION

The idea of Artificial Intelligence is to create a machine capable of rational thought and decisions like the human brain. The human brain receives most of its information using the percept of vision. For that reason it is of utmost importance to solve the problem of recognizing information about the outside world by an AI model similarly to human's sense of vision. Machine Learning is a subfield of AI that takes a step toward processing data obtained from the outside world, but lacks the ability to extract features from that data needed for accurate predictions. Deep Learning, which is a subfield of Machine Learning that has the ability to extract features, solves this problem and is the closest solution so far to being able to see the real world like humans do. One particular Deep Learning model that is the best suited for image recognition is the Convolutional Neural Network (CNN). While a Deep Learning model is a neural network containing one or more hidden layers made of neurons, CNN has preprocessing layers called convolutional layers that are able to extract features from images to prepare them for recognition by a deep neural network. CNN models have been proven to achieve high accuracies in image recognition. Our CNN model is tailored specifically toward recognizing American Sign Language letter images.

Sign Language is a precious tool developed for people who have hearing or speaking problems, which allows them to communicate. However, not everyone understands or knows Sign Language leading to the requirement of translation. Traditionally, this task is done by specially trained interpreters that may not always be available or present when needed. Having a machine that can perform translation of Sign Language would greatly improve life quality and communication for people who need it. Sign Language Recognition (SLR) is the machine recognition of hand signs in images or videos that outputs the corresponding meaning as text. SLR is not as mature of a field as other image recognition fields, such as Optical Character Recognition (OCR). However, the demand for it is high and the importance of its development is correspondingly strong. We hope that our work contributes to the development of SLR.

Most problems encountered in SLR correspond to general image recognition problems. One issue for image recognition is so called "noise" which means that the image contains unwanted details that obscure the contents of the image. When noise exists in an image, it is difficult for the SLR model to distinguish between hand signs and their surrounding background. Another problem is the similarity of different hand signs between each other that leads to the SLR model misrecognizing one letter for another. Even more problems include distortion and skew in images, low contrast between the meaningful part of the image and its background, distinguishing between hand signs and other objects in images, and recognizing hand signs under different lighting conditions and different viewing angles.

All of these problems, though not simple, are solvable using a CNN model. Such a solution takes advantage of the models ability to extract and learn features, which is the process of finding meaningful shapes and edges common to each hand sign letter class. This allows for accurate separation between the contents of the image and the background. Furthermore, the convolutional layer might contain filters that reduce the noise and improve image quality when necessary. Having a large enough dataset of images for training deals with the problem of different lighting conditions and viewing angles. Overall, CNN offers the techniques and algorithms needed to achieve high accuracy in Sign Language Recognition.

II. TASK DEFINITION

The goal of our project is to create an SLR program implementing a CNN model that will read an image containing a hand sign of a letter and output the letter in text format. The outputted letter should match the corresponding sign in the image with a high accuracy.

In our work, we created and tested an SLR program using a Deep Learning approach utilizing a CNN model. Our model has several convolutional layers responsible for preprocessing the image by extracting features and a neural network with a varying number of hidden layers. At first the model has to be trained, which is done through multiple iterations of forward passes and backpropagations. On each iteration the true labels are compared to the computational results obtained during the forward pass and the difference is calculated as a loss function. The computational results obtained during a forward pass depend entirely on the parameters called weights and biases. The gradients of the log loss function are calculated in order to minimize the loss. Then, the weights in both the convolutional and neural network layers are updated. Training continues until a minimal value of the loss function or the preset number of iterations is obtained. The end goal of model training is to obtain optimal values for the weights that consistently produce high accuracy recognition. After the training, the learned model is evaluated using images in the test dataset.

In general, the effectiveness of a model is determined by two critical parts: the model architecture and the data used to train and test the model. The dataset plays an especially important role in Deep Learning models, which highly depend on it. Deep Learning models are often the best choice to process big datasets with a large number of classes and amount of data itself. They also outperform other AI and Machine Learning models when processing datasets smaller in size but with more complex data. Images are good examples of big complex data since each image is defined not by one or several numbers but by a matrix of pixels, which may be very large depending on the image size and resolution. Video streams contain even more data as when processed they are split into a large amount of frames, which are static images. The scope of our model is limited to processing images, not videos. Correspondingly, we chose a dataset containing images of hand signs for training and testing purposes. This limitation is quite significant for SLR as two letters out of twenty-six in American Sign Language are represented by the movement of a hand, which would require video to recognize them. If our model demonstrates good results recognizing images we might consider extending it to process videos in prospective.

III. INFRASTRUCTURE

In order to implement the model, we chose to use Python as the language for our program, VS Code as our development environment, and GitHub for version control and information exchange between team members. To implement different tasks we used the following Python libraries: TensorFlow, Pandas, NumPy, OpenCV, and Matplotlib. TensorFlow contains ready to use functions to create a model, train it, specify model parameters, and test it. We used Pandas to read images from the dataset that are stored in csv spreadsheets. NumPy contains implementations of many widely used mathematical functions such as argmax. OpenCV is used for different operations with image files, such as reading, writing, and converting to different formats. Finally, Matplotlib is necessary to create graphs and plots. To train the model, we use two datasets - Sign Language MNIST dataset [1] and ASL Alphabet Dataset [2]. To test our model on unseen images an additional dataset, American Sign Language Hand Gesture Image Dataset [3], is used. The Sign Language MNIST dataset contains 27,455 training images and 7172 testing images that are in 28x28 pixel grayscale format. The ASL Alphabet Dataset contains nearly 230,000 images in 200x200 pixel color format. The American Sign Language Hand Gesture Image Dataset contains 36,000 high-resolution JPG images. In all three datasets, each image is a picture of a hand showing a letter in American Sign Language with variations corresponding to different lighting conditions, different backgrounds, varying viewing angles, etc. To accelerate the training we use a PC with a high performing GPU that TensorFlow can take advantage of.

IV. LITERATURE REVIEW

As a good starting point, paper [4] gives a comprehensive review of the history of SLR, its current status, and research papers that contributed to its development. Thus, it states that exploration into recognizing American Sign Language (ASL) first began with a research paper published in the mid-90s [5].

Since then, there have been a large number of research papers world-wide that studied SLR implementations for their corresponding languages. Deep Learning technologies opened a new era in the development of the field. Due to their capabilities of processing large complex data and extracting features, they offer incomparable opportunities for progress in SLR. In particular, CNN-based approaches have achieved state-of-the-art performance on several benchmark datasets [6]. During our research, we found three very recent papers related to our project.

The first paper [7] presents the results of the past five years of research into advancements, challenges, and opportunities in deep learning-based SLR. The authors explored such aspects of SLR as sign data acquisition technologies, sign language datasets, evaluation methods, and different types of neural networks. They come to the conclusion that Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) are the most effective models for static sign language recognition, which corresponds to the scope of our project. The paper provides a detailed description of a CNN's structure and components. The paper lists the main components of CNNs as convolutional layers, pooling layers, and fully connected layers. In our model we also use padding, a technique not mentioned in the paper. An image is fed into the input layer. Then the convolutional layer performs operations to extract local features. This is done in the following manner. A filter, also called a convolutional kernel, which is a small square matrix of numbers is slid over the input image and the dot product between the filter and the area it covers at the moment is computed. The result of applying a filter is a feature map, so the whole process corresponds to extracting features. The output of the convolutional layer is passed through a non-linear transformation using an activation function, most often ReLU (Rectified Linear Unit), to introduce non-linearity. The pooling layer reduces the size of the feature maps, retaining only important features. Then the output of the pooling layer becomes the input into the fully connected neural network. The last layer in the network is the output layer that produces the classification result. There are various datasets and models provided in the paper related to continuous rather than static inputs, which can be used for video stream recognition. This will be useful in our later work if we extend our model to processing videos. Together with the advances in SLR, the authors name challenges remaining in the field. The challenges mentioned in the paper include expanding sign language datasets, achieving user independence in recognition systems, improving semantic and syntactic understanding, and others. The paper concludes by emphasizing the necessity of further SLR development in order to address the previously discovered and newly emerging challenges.

The second paper [8] presents an evaluation of the performance of different deep neural network architectures in the recognition of sign language. From our point of interest, it references the research [9] that employed CNNs for feature extraction and classification. Guided by the utilization of pre-trained CNN models, like "AlexNet" and "VGG-16", the research described the remarkable achievement of an accuracy rate of up to 98% in sign language classification. Other research papers referenced in [8] also prefer CNN models for static image classification. Therefore, instead of comparing the CNN model to other AI models, the paper focuses on

comparing different architectures of CNN models. They mainly differ by the way hidden layers connect to each other. For example, Densenet [10], which is a CNN where every hidden layer is connected to all previous and future layers, was evaluated as the best architecture as it produced the highest recognition accuracy of 99%. Overall, the paper is a good source of methodologies to evaluate model effectiveness.

The third paper [11] provides a comprehensive overview of models, methodologies, and challenges in SLR based on the 32 research papers that the authors studied and reviewed. The paper mostly focuses on continuous sign language recognition (CSLR) that we don't use at the current phase of our project but may consider using in the future. In particular, the paper found the following limitation and research gaps that serve as pointers for future developments in the field. The main limitations come from the lack of existing datasets with high quality data and the difficulties of creating new ones that require a lot of resources and expenses. The authors point out to the necessity of developing models capable of working efficiently with small training datasets. Another suggested solution is to develop techniques for data generation, such as Generative Adversarial Networks (GANs). Other limitations relate to conditional changes during the gathering of data and testing, such as different signers producing variations in hand signs. Again, we don't currently address these challenges in our model since it is not continuous SLR, but it is good to know about them considering our future plans.

V. APPROACH

A. Our CNN Model

The input layer to our model contains a number of nodes that correspond to the number of pixels in the input image. This is defined by the image format in the dataset that we will describe later. From the input layer the data feeds into the convolutional layer where feature extraction from the image occurs using padding, filtering, and pooling. The padding adds pixels with 0 values around the perimeter of the input image in order for the image to retain its size after filtering. Filtering is done in order to detect patterns or features, such as edges, corners, and curves. This highlights the important shapes in the image and separates them from background and other meaningless objects that significantly improve recognition. At first, we used two sets of 3x3 dynamic filters. The first set of 32 filters detected simple features (edges, lines). The second set of 64 filters detected more complex patterns (shapes, curves, and letters). The filters are called dynamic because their matrices are not set, but change during model training the same way as the weights in the fully connected layer. In the final model we added a third set of 128 dynamic filters and the following static filters: Gaussian, Sobel X, Sobel Y, and Laplacian. Each filter is slid across an image moving with a stride of 1. The stride of the filter indicates how many pixels to the right and down the filter moves on each step of computation. The result of the computation is a feature map and the computation itself uses dot multiplication shown by formula (1) in the Equations section. Based on the purpose of the filter they are represented by different matrices. When these matrices are multiplied with part of an image, the resulting feature map is specific to each type of filter. Thus,

the Gaussian filter reduces noise in an image. The Sobel X filter detects vertical edges. The Sobel Y filter detects horizontal edges. The Laplacian filter also detects edges but is direction independent and is able to detect both sides of an edge. In the dynamic filters the ReLU activation function described by formula (2) is applied to the feature map in order to add non-linearity to the features learned. Since the resulting feature map is the same size as the input image, in most cases it is unnecessarily large and contains both important and not important features. To retain only important features in the map, pooling is used. Pooling is done using kernels similar to filters, but smaller in size, which slide across the feature map and compress the map. We use the MaxPooling function that builds a new compressed feature map by selecting a pixel with the largest value inside each area that it slides over. In order to demonstrate the feature map produced as a result of the application of each filter, we show the feature map obtained from static filters and use the GradCam technique to visualize the features extracted by the dynamic filters. This can be seen in the figures in the Results section.

From the convolutional layer the data is passed to the fully connected layer. At first, we used one hidden layer containing 128 neurons and a multi-class output layer. In the final model we added one more hidden layer with 256 neurons. In a hidden layer all neurons are connected to every input and every output of the layer. Each neuron takes the input from the previous layer as a vector, computes the result using ReLU activation function described by formula (2), and sends it to every neuron in the next layer. In order to diminish overfitting during training, both hidden layers contain dropout that randomly turn off a certain percentage of neurons on every iteration. The output layer contains 26 neurons which correspond to 26 possible output classes. Each class represents a letter in the English alphabet. The resulting output is a vector of size 26 with each place in the vector containing the probability of the corresponding class. This vector is obtained using the Softmax activation function described by formula (3).

At first the model goes through training over a number of iterations (epochs), which we set to 15. In each iteration the input image goes through all of the layers described above in a forward pass and ends up with a predicted class. This result is compared to the true class specified in the training image label as a one-hot encoded vector. Next, the predicted result and the label are used to compute the log loss function, also known as categorical cross-entropy loss, described by formula (4). The goal of the training is to find weight and bias parameters, such that the loss function is minimized. This is achieved by calculating the gradients of the loss function with respect to the weights and biases. The gradients are found by calculating partial derivatives over weights, biases, and results of all layers using the chain rule starting from the output layer and going back to the input layer. This process is known as backpropagation [12]. After a forward pass and backpropagation the weights and biases are updated in all layers including the dynamic convolutional layers and the new values are used in the next iteration. How the new weights and biases values are obtained is described by formulas (5) and (6). In our model the training is completed after 15 iterations. For efficiency, our model processes input images as batches of 64 at the same time in parallel.

The last part of our model is the evaluation phase that

predicts the classes of images in the testing dataset. Each prediction is compared to the label of the corresponding image in order to calculate the overall accuracy of the model as described by formula (7). The evaluation phase only goes through the forward pass and does not use backpropagation.

B. Equations

On each stride during filtering the feature map is calculated by the formula:

$$y = \sum_{i=1}^m \sum_{j=1}^n X_{i,j} \cdot W_{i,j} + b \quad (1)$$

In this formula $\mathbf{W}$ is a matrix for the filter kernel that is multiplied using the dot product with the $\mathbf{X}$ matrix representing the area of the image covered by the filter at the current stride. b is the number representing the bias for each filter needed to avoid zero values. This formula describes the per element multiplication that is the number in the filter multiplied by the number in the image in the corresponding matrix position and then all of the products are summed together to produce the new matrix that is the feature map.

The computation in each neuron in hidden and convolutional layers is performed using the ReLU activation function below that takes $\mathbf{W} \cdot \mathbf{X} + \vec{b}$ as the input parameter:

$$\text{ReLU}(x) = \begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases} \quad (2)$$

In this formula x is replaced with $\mathbf{W} \cdot \mathbf{X} + \vec{b}$ where $\mathbf{W}$ is the weights matrix, $\mathbf{X}$ is the matrix representing the feature map, and $\vec{b}$ is the biases vector. ReLU outputs $\mathbf{W} \cdot \mathbf{X} + \vec{b}$ if it is greater than 0 and outputs 0 otherwise.

The computation in each neuron in the output layer is performed using the Softmax activation function below that takes $\mathbf{W} \cdot \mathbf{X} + \vec{b}$ as the input parameter:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \quad (3)$$

In this formula $z_i = W_i x + b_i$ in the output node i where x is the output from the hidden layer, W_i is the weight of the node, and b_i is the bias of the node. The denominator in the formula is the sum of e to power of every $z_j = W_j x + b_j$. $\hat{y}_i$ is the probability of the class at node i . C is the number of classes which is 26 in our case. 26 output nodes produce a result in the form of a vector with the probabilities of each class at the index of each class.

The log loss function is described by the formula:

$$L = - \sum_{i=1}^C y_i \cdot \log(\hat{y}_i) \quad (4)$$

In this formula y_i is the value at index i of the one-hot encoded vector. $\hat{y}_i$ is the probability of the class at index i in the output vector. C is the number of classes. L is the loss.

The new values for the weights and biases in all layers are obtained using the formulas:

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w} \quad (5)$$

$$b_{new} = b_{old} - \eta \frac{\partial L}{\partial b} \quad (6)$$

In these formulas w_{new} and b_{new} are the new values of weights and biases, w_{old} and b_{old} are the old values, η is the learning rate, and L is the loss.

The accuracy of the model is calculated using the formula:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N 1(\hat{y}_i = y_i) \quad (7)$$

In this formula N is the total number of images in the train or test set. y_i is the true class label for image i . $\hat{y}_i$ is the predicted class label by the model. $1(\hat{y}_i = y_i)$ is an indicator function that is 1 if the prediction is correct and 0 if it is wrong. The Accuracy represents the fraction of correctly predicted images by the model out of all evaluated images.

C. Training and Testing Datasets

As mentioned in the Infrastructure section, we used the Sign Language MNIST dataset and the ASL Alphabet Dataset from Kaggle to train our model. Two different datasets were used in order to evaluate the dependency of the model on the training dataset. The MNIST dataset contains two spreadsheets with 27,455 training images and 7172 testing images. Each row in the spreadsheets contains the label of the image in the first column and the image pixel values in the next 784 columns. Each pixel value is in the range from 0 to 255. This corresponds to an image of 28 x 28 pixels in grayscale format. This dataset is a result of collecting hand sign images of each letter in the American Sign Language without any images of the letters j and z as they require hand motion. The ASL Alphabet Dataset is structured differently. The images corresponding to each letter are gathered in a corresponding folder, with about 8000 images per letter. Each image is 200x200 pixels in the JPEG format. Grayscale vs. color images determine the number of input channels to the convolutional layer. There is one channel for grayscale images and three channels for color images. Unlike the MNIST dataset, the ASL Alphabet Dataset is not split into training and testing parts. Therefore, before training with that dataset, we split it. We used the testing part of the split datasets to evaluate our model after training. In order to use the model in the real world, it has to perform well on images that it was not trained on. For that purpose, in addition to evaluating the model on the testing parts of the training datasets, we test it on unseen images. These new images are obtained from a third dataset, American Sign Language Hand Gesture Image Dataset. This dataset is structured similarly to the ASL Alphabet Dataset. That is, it contains hand sign images representing each letter in corresponding folders. The format of the images is also JPEG; however, the number of images for each letter is only 1000. It is important that we used the same testing dataset for each training dataset. This provided a fair comparison as the testing dataset acted as a control variable. Besides evaluating the model's dependency on the format of the training dataset, we also evaluated its dependency on the size of the training dataset. Having the large ASL Alphabet Dataset with about 8000 images per letter allowed to create different models trained on a varied number of images. In total we trained three models, one with the MNIST dataset, the second one with 1000 images per letter from the ASL Alphabet Dataset, and the third one with 2000 images per letter from the ASL Alphabet Dataset. All three models were evaluated using the testing part of their corresponding training dataset and tested on the images from the American Sign Language Hand Gesture Image Dataset. In all three datasets that we used for training and testing, the images of each hand sign are varied by different lighting conditions, different backgrounds, different viewing angles of the hand sign, etc. Since we tested the models on a

dataset other than the training dataset, it was important to convert the testing images into the format of the training images prior to inputting them into the CNN. For example, images from the American Sign Language Hand Gesture Image Dataset were converted from their large size color JPEG format to the 28x28 pixel grayscale format when testing the model trained with the MNIST dataset. For all three models, the results demonstrated extremely high accuracy during evaluation and poor accuracy during testing as shown in the Results section.

VI. RESULTS

The goal of our project is to obtain a CNN model able to recognize a letter from a hand sign image with a high accuracy. The performance of such a model depends on several factors. The key factors are the architecture of the model, the values of the hyperparameters, and the dataset used for training. The architectures of the initial and final models were discussed in the Approach section. As it was pointed out there, the final model was upgraded from the initial one by adding two convolutional layers, one with 4 different static filters and one with 128 dynamic filters, and an additional hidden layer with 256 neurons. This change resulted in the evaluation accuracy increasing from around 90% to around 98% as will be seen later in this section. The introduction of static filters influenced the speed of learning to a larger degree than the final evaluation accuracy of the model. After tuning the hyperparameters we ended up with the following values: number of epochs = 15, learning rate = 0.001 (default for the Adam optimizer), batch size = 64, number of filters in the first dynamic convolutional layer = 32, number of filters in the second dynamic convolutional layer = 64, number of filters in the third dynamic convolutional layer = 128, number of neurons in the first hidden layer = 256, the number of neurons in the second hidden layer = 128, dropout rate in the first hidden layer = 0.7, and dropout rate in the second hidden layer = 0.5. As discussed in the Approach section, we obtained three models trained with different datasets and in this section we will show and compare the results for each model.

A. MNIST Model

The model trained on the MNIST dataset produced the following results. After 15 epochs the final training loss was 0.0249, the validation loss was 0.0459, the training accuracy was 0.9926, and the evaluation accuracy was 0.9856. The decrease of the loss function during training and validation phases is shown on Figure 1 below:

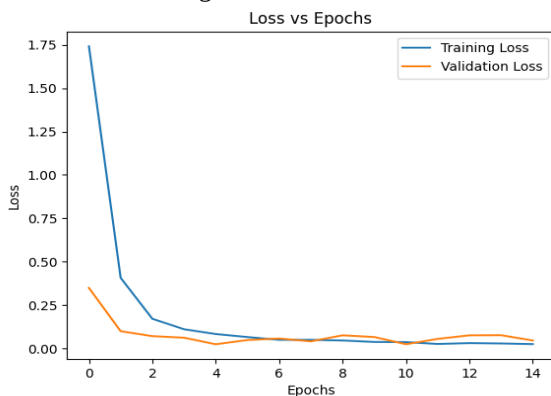


Fig 1. Training and Validation loss for MNIST Dataset.

The increase of the accuracy of the model during training and validation phases is shown on Figure 2 below:

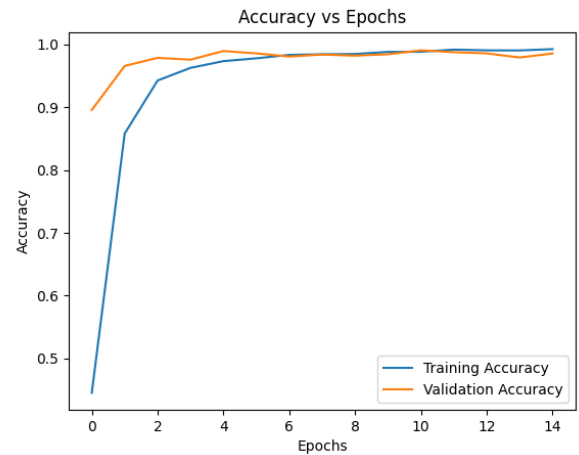


Fig 2. Training and Validation accuracy for MNIST Dataset.

Prediction results of the first 12 images in the validation set and their corresponding labels are shown on Figure 3 below:

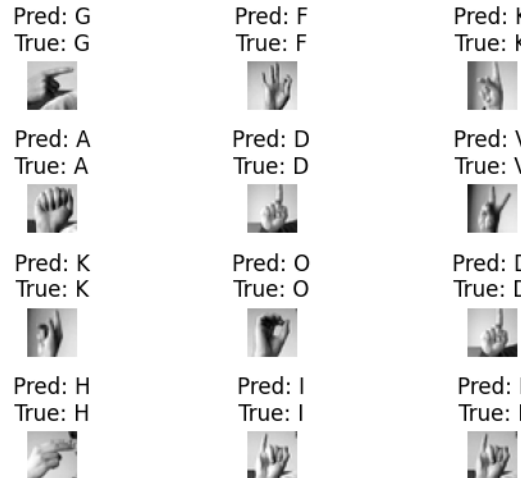


Fig 3. The predictions and labels of the first 12 validation images.

The full comprehensive results of the validation phase can be seen in the confusion matrix shown on Figure 4 below:

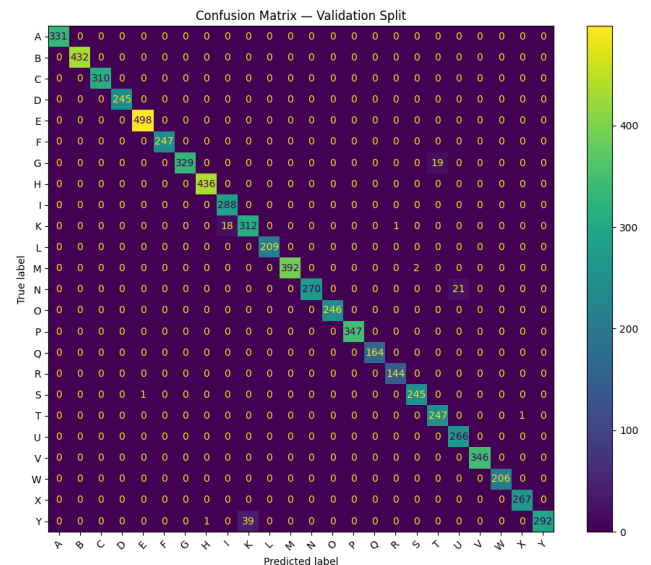


Fig 4. Confusion matrix for the validation image set.

As can be seen on Figure 3, all 12 sample validation images were correctly identified. The confusion matrix in Figure 4 shows the negligibly small number of mismatches compared to the correct predictions during validation. These results support the high validation accuracy of 99% stated earlier.

Despite the promising validation results, the testing accuracy on 24 images (one image per letter excluding j and z) from the American Sign Language Hand Gesture Image Dataset was 29.2%. The correctly recognized images are indicated with green text and the incorrectly ones are indicated with red text on Figure 5 below:



Fig 5. Sample predictions with 7 out of 24 correct during testing.

The results of the testing phase can be seen in the confusion matrix shown on Figure 6 below:

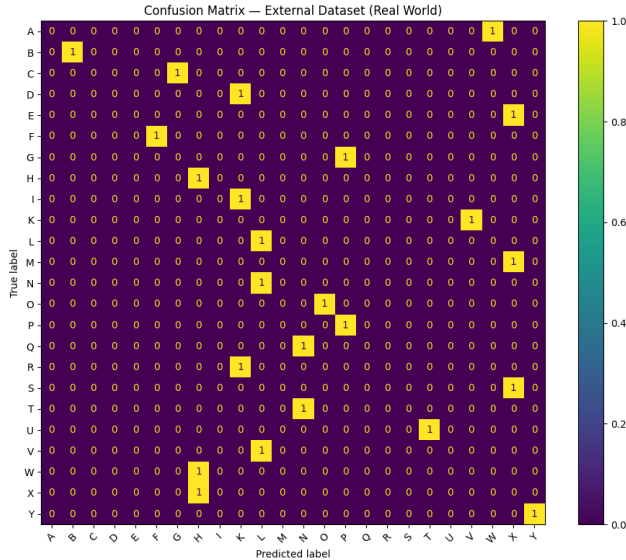


Fig 6. Confusion matrix for the testing image set.

Such a large difference between the validation and testing accuracies is addressed in the Error Analysis section.

In order to gain a better understanding of the inner workings of our CNN model, we obtained visualizations of the results of image processing in each convolutional layer. Figures 7 and 8 show the features extracted from the image after applying each of 4 static filters. The first image on Figure 7 is the original input image converted to MNIST format. The next two images are feature maps extracted after applying Gaussian and Sobel X filters correspondingly. The two images on Figure 8 show the feature maps after applying

Sobel Y and Laplacian filters.

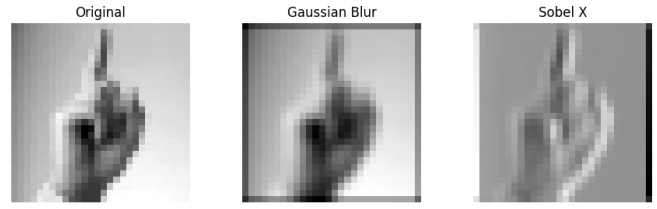


Fig 7. The result of applying the first two static filters.

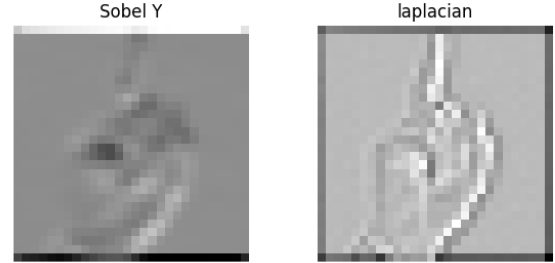


Fig 8. The result of applying the last two static filters.

Figure 9 demonstrates the features learned after applying filters in each of the three dynamic convolutional layers. The images are ordered from left to right in correspondence with the layers.

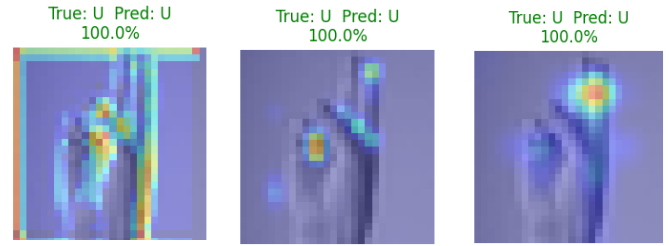


Fig 9. Features learned in three convolutional layers ordered left to right.

These visualizations were obtained using the GradCam technique that highlights regions that most influence the model's predictions.

B. ASL Alphabet Dataset 1000 Images Model

This subsection demonstrates the results obtained for the model trained on the ASL Alphabet Dataset with 1000 images per letter. After 15 epochs the final training loss was 0.4079, the validation loss was 0.1503, the training accuracy was 0.8644, and the evaluation accuracy was 0.9617. The decrease of the loss function during training and validation phases is shown on Figure 10 below:

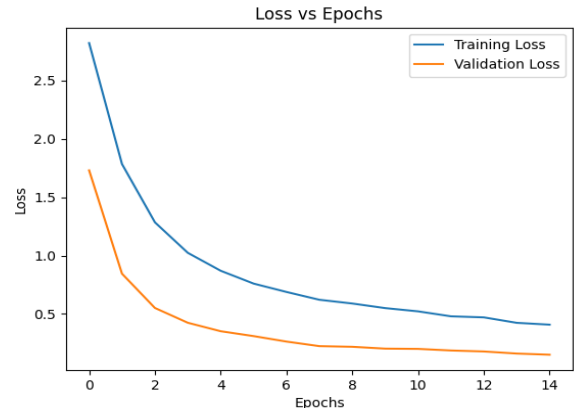


Fig 10. Training and Validation loss for ASL Alphabet Dataset 1000.

The increase of the accuracy of the model during training and validation phases is shown on Figure 11 below:

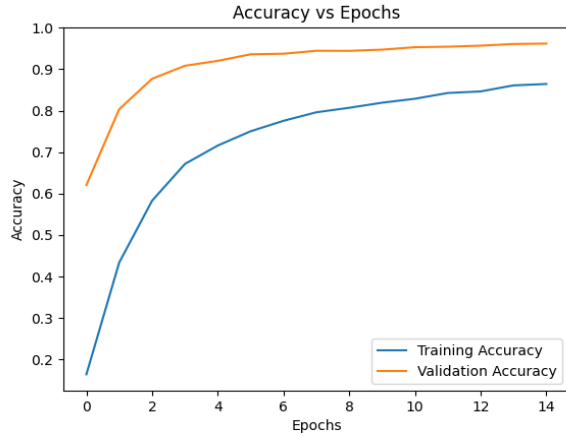


Fig 11. Training and Validation accuracy for ASL Alphabet Dataset 1000.

Compared to the MNIST model, the training loss is visibly higher than the validation loss and, correspondingly, the training accuracy is visibly lower than the validation accuracy. This can be explained by the differences between the datasets with the ASL Alphabet Dataset having much more variety in image contents. Prediction results of the first 12 images in the validation set and their corresponding labels are shown on Figure 12 below:



Fig 12. The predictions and labels of the first 12 validation images.

Confusion matrix shows the validation results on Figure 13:

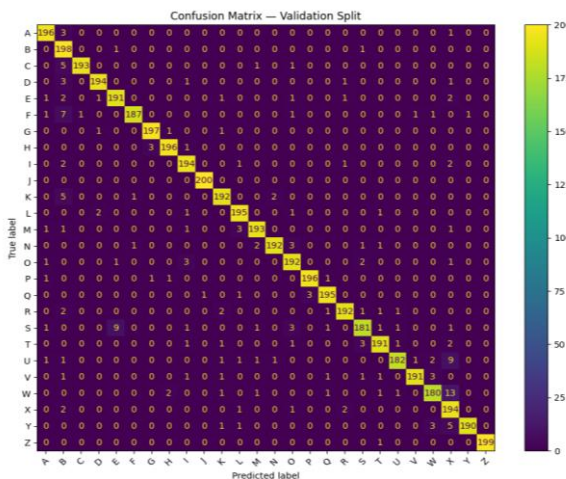


Fig 13. Confusion matrix for the validation image set.

Again, Figure 12 shows that all 12 sample validation images were correctly identified and Figure 13 shows a nearly perfect confusion matrix. These results again support the high validation accuracy of 96% stated earlier.

Similarly to the validation results, the testing results were also very close to the ones obtained with the MNIST model. The testing accuracy on 24 images (one image per letter excluding j and z) from the American Sign Language Hand Gesture Image Dataset was again 29.2%. The correctly recognized images are indicated with green text and the incorrectly ones are indicated with red text on Figure 14 below:

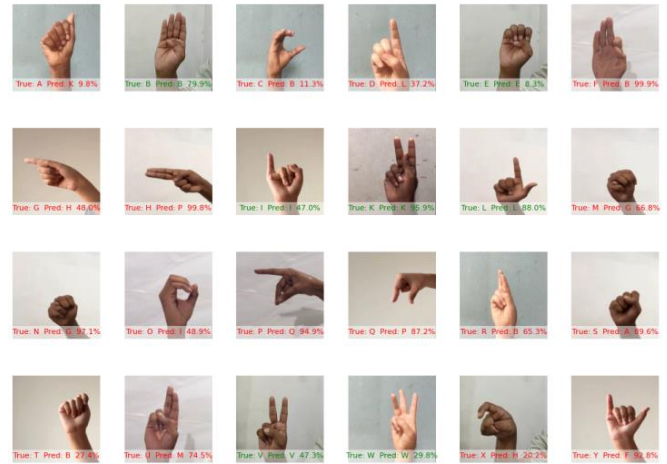


Fig 14. Sample predictions with 7 out of 24 correct during testing.

The results of the testing phase can be seen in the confusion matrix shown on Figure 15 below:

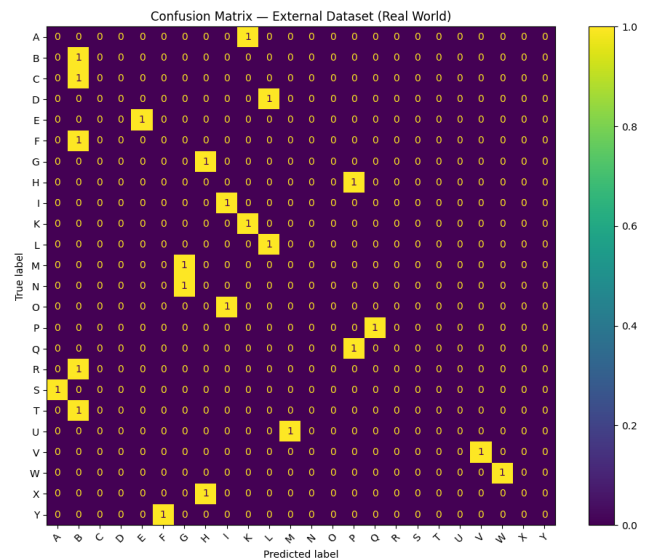


Fig 15. Confusion matrix for the testing image set.

The visualizations of the results of image processing in each convolutional layer are shown in Figures 16 and 17. The first image on Figure 16 is the original input image resized to 64x64 pixels. The next two images are feature maps extracted after applying Gaussian and Sobel X filters correspondingly. The two images on Figure 17 show the feature maps after applying Sobel Y and Laplacian filters. Figure 18 demonstrates the features learned after applying filters in each of the three dynamic convolutional layers. The images are ordered from left to write in correspondence with the layers.



Fig 16. The result of applying the first two static filters.

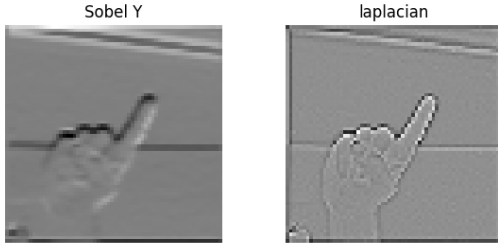


Fig 17. The result of applying the last two static filters.



Fig 18. Features learned in three convolutional layers ordered left to right.

C. ASL Alphabet Dataset 2000 Images Model

This subsection demonstrates the results obtained for the model trained on the ASL Alphabet Dataset with 2000 images per letter. After 15 epochs the final training loss was 0.2246, the validation loss was 0.0855, the training accuracy was 0.9288, and the evaluation accuracy was 0.9763. The decrease of the loss function during training and validation phases is shown on Figure 19 below:

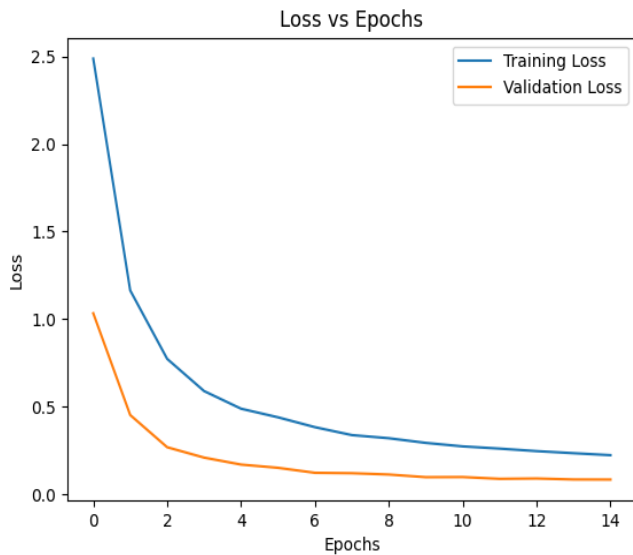


Fig 19. Training and Validation loss for ASL Alphabet Dataset 2000.

The increase of the accuracy of the model during training and validation phases is shown on Figure 20 below:

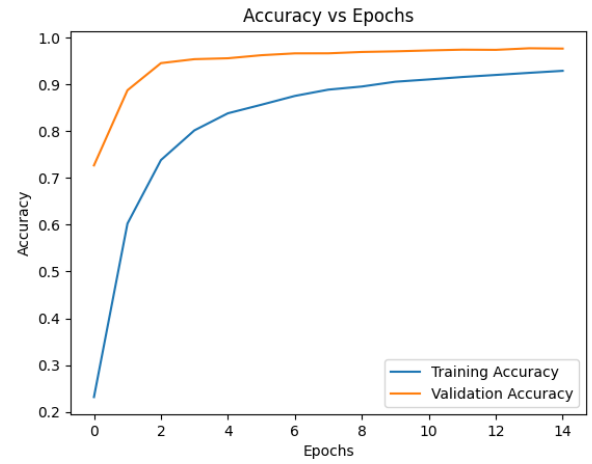


Fig 20. Training and Validation accuracy for ASL Alphabet Dataset 2000.

As expected, the training and validation loss were lower and the training and validation accuracies were higher when the model was trained on 2000 images per letter compared to 1000 images. At the same time, the training and validation results were better in the MNIST model. Prediction results of the first 12 images in the validation set and their corresponding labels are shown on Figure 21 below:

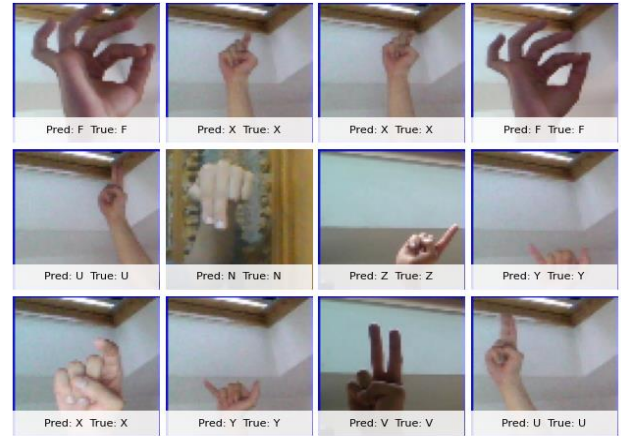


Fig 21. The predictions and labels of the first 12 validation images.

Confusion matrix shows the validation results on Figure 22:

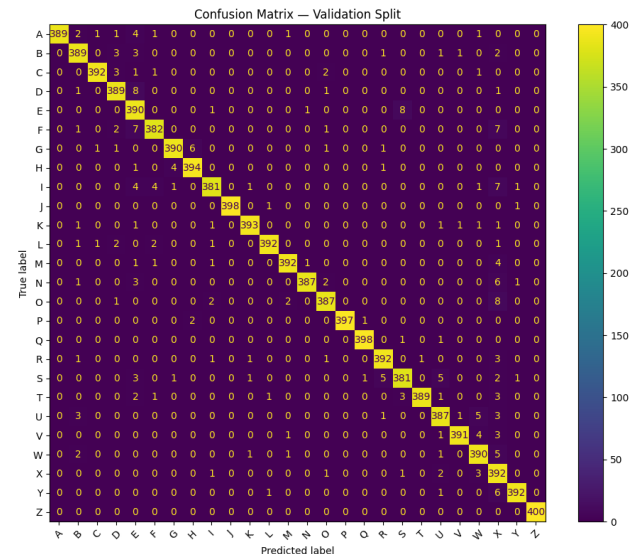


Fig 22. Confusion matrix for the validation image set.

Once again, Figure 21 shows that all 12 sample validation images were correctly identified and Figure 22 shows a nearly perfect confusion matrix. These results again support the high validation accuracy of 98% stated earlier.

Surprisingly, the testing results appeared to be even worse than the ones from the previous two models. The testing accuracy on 24 images (one image per letter excluding j and z) from the American Sign Language Hand Gesture Image Dataset was 20.8%. The correctly recognized images are indicated with green text and the incorrectly ones are indicated with red text on Figure 23 below:

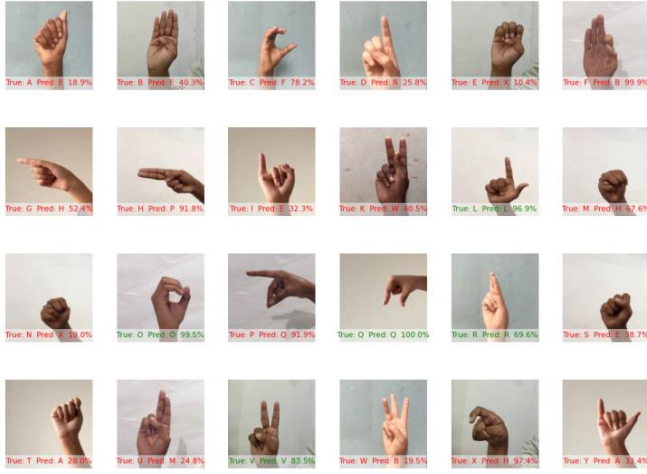


Fig 23. The predictions and labels of the first 12 validation images.

The results of the testing phase can be seen in the confusion matrix shown on Figure 24 below:

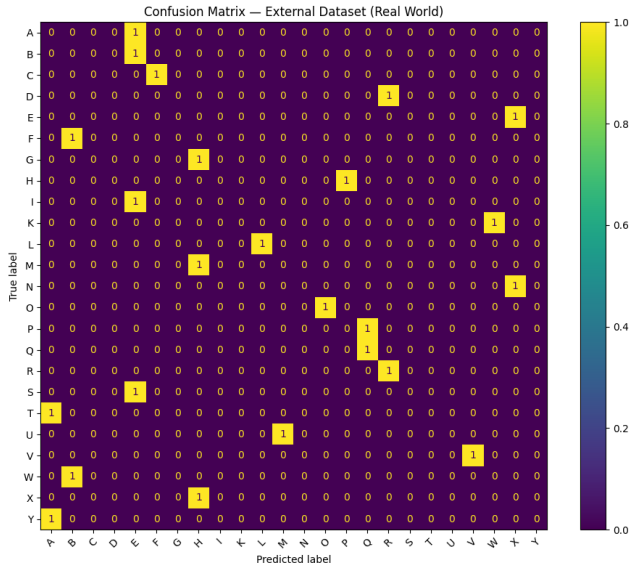


Fig 24. Confusion matrix for the testing image set.

The visualizations of the results of image processing in each convolutional layer are shown in Figures 25 and 26. The first image on Figure 25 is the original input image resized to 64x64 pixels. The next two images are feature maps extracted after applying Gaussian and Sobel X filters correspondingly. The two images on Figure 26 show the feature maps after applying Sobel Y and Laplacian filters. Figure 27 demonstrates the features learned after applying filters in each of the three dynamic convolutional layers. The images are ordered from left to write in correspondence with the layers.



Fig 25. The result of applying the first two static filters.

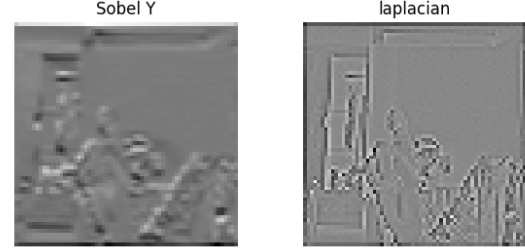


Fig 26. The result of applying the last two static filters.



Fig 27. Features learned in three convolutional layers ordered left to right.

To summarize, all three models demonstrated similar results with very high training and validation accuracies and poor testing accuracies. We explain these results in the Error Analysis section.

VII. ERROR ANALYSIS

Comparing the recognition performances of the three models, the following can be said. A CNN model can achieve nearly perfect accuracy when recognizing images from the sets the model was trained on. This is because the model memorizes the important features in conjunction with less important features. In other words, it does not only learn what letter a particular hand sign corresponds to but also learns the environment surrounding the hand sign. This can be seen in the GradCam visualizations as besides highlighting parts of the hand, it also highlights parts of the surroundings. This also explains the poor recognition on images not from the training set. The model has difficulties focusing only on the hand, which is the only object in the image that relates to letter recognition. Instead, the model treats the hand and its surroundings together and tries to use all of this information during recognition. Our expectation is that significantly increasing the training set including a great variety of all possible combinations of hand signs and environments would increase the testing accuracy. In line with this expectation, we increased the training dataset from 1000 images per letter to 2000 images per letter. However, this was not enough and actually caused a small decrease in accuracy. This can be explained by model overfitting also increasing when increasing the training dataset. We still suppose that increasing the number of images per letter to 8000 or even

10,000 would prove the expectations correct. We were not able to test this because even with 4000 images per letter the program execution ran out of memory on both our own PC and Google Colab. This makes sense as the CNN must load a large number of the training images, which are entire pixel matrices, at once in memory to perform training. It is known that CNN models show most of their potential when processing extremely large data. Correspondingly, when the training dataset is very large, the recognition performance should be much better. The memory limitation that we discovered makes this harder to prove without access to specialized hardware.

The “Recent Advances on Deep Learning for Sign Language Recognition” by Zhang et al. [7] compares SLR performed by different Deep Learning models. It highlights CNN models as one of the best fits for SLR capable of achieving nearly 100% recognition. At the same time, the paper does not specify if these results were achieved when recognizing images from the same dataset that the model was trained on. The concrete training dataset sizes are also not specified. Therefore, if the results presented in the paper were achieved on the training datasets, then this is in line with our results. Otherwise, if the results were achieved on unseen images, we assume that very large training datasets were used. As stated earlier, we could not use very large training datasets in our projects but the authors presenting the results in the paper most likely had access to more advanced hardware. However, the CNN constraints pointed out in the paper, such as limited availability of large-scale and diverse datasets, the contradiction between model accuracy and computational power, real-time processing and latency, and others, completely coincide with the limitations we experienced.

VIII. FURTHER DEVELOPMENT

After implementing the initial model, we planned to add static filters, tune hyperparameters for the best performance, try different datasets for training, and test the model on new images. We were able to complete those plans in the final model as described in this report. The more ambitious plan to apply the model to video streams instead of static images still remains a direction for further improvement. Overall, most of the changes suggested and implemented brought about positive results.

IX. CONCLUSION

In conclusion, we have successfully implemented three CNN models, differing by training dataset, capable of recognizing hand sign images for letters with high accuracy. Our results demonstrated a very good model performance when recognizing images from the dataset the model was trained on. When recognizing images not from the training dataset the models performed poorly. To improve this, training datasets of much larger sizes are required. However, the memory limitations we discovered when trying to use them with our models prevented proving this assumption. We compared three models to study a CNN model’s dependency on the format and size of the training dataset. The comparison showed very similar results with validation accuracy in the range of 96% to 99%. The testing accuracy was similarly poor

for both the MNIST model and the ASL Alphabet Dataset 1000 images model and even worse for the ASL Alphabet Dataset 2000 images model due to increased overfitting. In addition to computing and plotting the final results, we looked deeper into the inner workings of CNN models by visualizing the features extracted in each convolutional layer and understanding the effect each layer has during recognition. Overall, we are satisfied with our implementation of our CNN models, obtained results, and goals achieved.

X. CONTRIBUTION

The contribution of each team member is shown below:

Objectives	Contributor(s)
Topic selection and proposal	Michael and Robert
GitHub repository	Robert
Dataset selections	Michael
CNN architecture	Michael and Robert
Model implementation	Robert
Progress report	Michael and Robert
Model training/validation	Michael
Model improvements	Robert
Testing on unseen images	Michael
Final Report	Michael and Robert

REFERENCES

- [1] “Sign Language MNIST,” [www.kaggle.com. https://www.kaggle.com/datasets/datamunge/sign-language-mnist/data](https://www.kaggle.com/datasets/datamunge/sign-language-mnist/data).
- [2] Debashish Sau, “ASL (American Sign Language) Alphabet Dataset,” Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/debashishsau/aslamerican-sign-language-aplphabet-dataset>
- [3] F. Islam, “ASL-HG: American Sign Language Hand Gesture Image Dataset,” Mendeley Data, vol. 1, Nov. 2025, doi: <https://doi.org/10.17632/j4y5w2c8w9.1>.
- [4] A. Abdullah, G. A. Amoudi, and H. S. Alghamdi, “Advancements in Sign Language Recognition: A Comprehensive Review and Future Prospects,” IEEE Access, vol. 12, pp. 128871–128895, Jan. 2024, doi: <https://doi.org/10.1109/access.2024.3457692>.
- [5] U. Farooq, M. S. M. Rahim, N. Sabir, A. Hussain, and A. Abid, “Advances in machine translation for sign language: Approaches, limitations, and challenges,” Neural Comput. Appl., vol. 33, no. 21, pp. 14357–14399, Nov. 2021.
- [6] A. A. Barbhuiya, R. K. Karsh, and R. Jain, “CNN based feature extraction and classification for sign language,” Multimedia Tools Appl., vol. 80, no. 2, pp. 3051–3069, Jan. 2021.
- [7] Y. Zhang and X. Jiang, “Recent Advances on Deep Learning for Sign Language Recognition,” Computer Modeling in Engineering & Sciences, vol. 139, no. 3, pp. 2399–2450, 2024, doi: <https://doi.org/10.32604/cmescs.2023.045731>.
- [8] H. M. Sarmad Khan, S. D. McLoughlin, and I. Murtagh, “Comparative Evaluation and Utilization of Convolutional Neural Network Architectures for Irish Sign Language Recognition,” International Journal of Combinatorial Optimization Problems and Informatics, vol. 16, no. 1, pp. 123–131, Mar. 2025, doi: <https://doi.org/10.61467/2007.1558.2025.v16i1.550>.
- [9] A. A. Barbhuiya, R. K. Karsh, and R. Jain, “CNN based feature extraction and classification for sign language,” Multimedia Tools Appl., vol. 80, no. 2, pp. 3051–3069, 2021, doi: [10.1007/S11042-020-09829-Y](https://doi.org/10.1007/S11042-020-09829-Y).
- [10] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, “Densely connected convolutional networks,” in Proc. 30th IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2017, pp. 2261–2269, doi: [10.1109/CVPR.2017.243](https://doi.org/10.1109/CVPR.2017.243).
- [11] H. A. Taher and S. R. M. Zeebaree, “A Critical Study of Recent Deep Learning-Based Continuous Sign Language Recognition,” The Review of Socionetwork Strategies, vol. 19, no. 1, pp. 131–161, 2025, doi: [10.1007/s12626-025-00180-y](https://doi.org/10.1007/s12626-025-00180-y).
- [12] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” Nature, vol. 323, no. 6088, pp. 533–536, Oct. 1986, doi: <https://doi.org/10.1038/323533a0>.